

其中, q_i 为无关问题的查询向量, β 为指定的激活阈值, γ 为指定的激活值差距的阈值。具体而言, 第一项 l_{m1} 负责确保补丁不会对无关的输入进行激活。这通过对每个无关输入的查询向量 q_i 的激活值进行阈值限制实现, 若激活值超过阈值 β 则会产生惩罚。而第二项 l_{m2} 旨在放大补丁对目标查询向量 q_e 和无关查询向量 q_i 的激活值差距。这通过要求目标查询向量的激活值显著高于所有无关查询向量的激活值的最大值来实现。

将这些损失项整合, T-Patcher 的总损失函数 L_p 可以表达为:

$$L_p = L_{Acc} + \beta \cdot L_m = l_e + \alpha \cdot l_a + \beta \cdot (l_{m1} + l_{m2}), \quad (5.14)$$

其中, β 是记忆损失项的权重。通过这种设计, T-Patcher 不仅可以确保补丁正确地修改模型的输出, 还能减少对其他问题的影响, 从而实现准确且可靠的模型编辑。

尽管 T-Patcher 实现了模型的精确调整, 在 GPT-J 模型上的准确性和泛化性较好, 但是一些研究表明 [27], 它也存在一些局限性。例如, 在不同模型架构上, 其性能会有所波动; 在批量编辑时, 对内存的需求较高, 可能限制其在资源受限环境下的应用。相比之下, ROME 方法表现得更加稳定。它将知识编辑视为一个带有线性等式约束的最小二乘问题, 从而实现对模型特定知识的精确修改, 在编辑的准确性、泛化性和局部性等方面都表现出色。该方法将在第 5.4 节介绍。

5.4 定位编辑法: ROME

定位编辑首先定位知识存储在神经网络中的哪些参数中, 然后再针对这些定位到的参数进行精确的编辑。ROME (Rank-One Model Editing) [17] 是其中的代表性方法。本节将详细介绍 ROME 的知识定位过程及相应的编辑方法。

5.4.1 知识存储位置

大脑的记忆存储机制一直是人类探索的谜题。这一问题不仅限于脑科学领域，对于具有强大智能的大语言模型，其知识存储及回忆机制也亟待研究。要解决这个问题，首先要定位出大语言模型的知识存储在哪些参数中，即存储的位置。通过对知识进行定位，可以揭示模型内部的运作机制，这是理解和编辑模型的关键步骤。ROME 通过因果跟踪实验和阻断实验发现知识存储于模型中间层的全连接前馈层。

1. 因果跟踪实验

ROME 采用控制变量的策略，首先对模型的推理过程实施干扰，然后进行局部恢复并观察影响，探究模型中不同结构与具体知识在推理过程中的相关性，从而确定知识在模型中的具体位置。该实验被称为**因果跟踪**，包含三个步骤：**正常推理**、**干扰推理**和**恢复推理**。其中，**正常推理**旨在保存模型在未受干扰情况下的内部状态，用于后续恢复推理中内部状态的恢复；**干扰推理**旨在干扰模型的所有内部状态，作为控制变量的基准线；**恢复推理**则将每个内部状态的恢复作为变量，通过对比内部状态恢复前后的输出差异，精确评估每个模块与知识回忆的相关性。

因果跟踪实验针对**知识元组**进行研究。在这个实验中，每个知识被表示为知识元组 $t = (s, r, o)$ ，其中 s 为**主体**， r 为**关系**， o 为**客体**。例如，“斑马的肤色是黑色”可以表示为知识元组：（“斑马”，“的肤色是”，“黑色”）。此外，模型的输入问题为 $q = (s, r)$ ， $q^{(i)}$ 表示 q 的第 i 个 Token。我们期望模型在处理问题 q 时能够输出对应的客体 o 作为答案。具体地，因果跟踪实验的步骤如下：

1. **正常推理**：将 q 输入语言模型，让模型预测出 o 。在此过程中，保存模型内部的所有模块的正常输出，用于后续恢复操作。
2. **干扰推理**：向 s 部分的嵌入层输出添加噪声，破坏其向量表示。在这种破坏输入的情况下，让模型进行推理，在内部形成被干扰的混乱状态。

3. **恢复推理**：在干扰状态下，对于输入问题的每一个 Token $q^{(i)}$ ，将 $q^{(i)}$ 在每一层的输出向量分别独立地恢复为未受噪声干扰的“干净”状态，并进行推理。在每次恢复时，仅恢复一个特定位置的输出向量，其余内部输出仍保持干扰状态。之后，记录模型在恢复前后对答案的预测概率增量，该增量被称为模块的**因果效应**，用来评估每个模块对答案的贡献。

以问题“斑马的肤色是”为例，其因果跟踪过程如下：

当输入问题“斑马的肤色是”时，模型会推理出答案“肉色”（假设该模型不知道正确答案是黑色）。此时，保存所有模块在正常推理过程中的输出，见图 5.12。

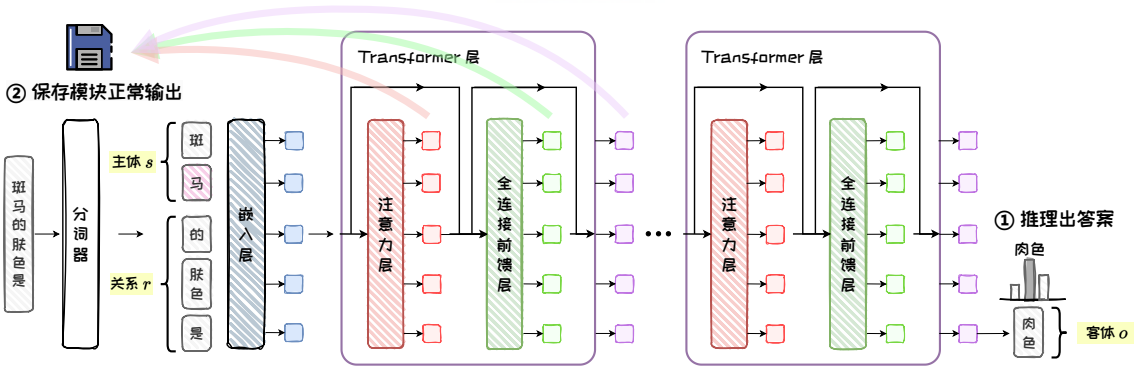


图 5.12: 正常推理。

然后，在嵌入层对 $s =$ “斑马” 的每个 Token 的嵌入向量添加噪声，接着在噪声干扰下进行推理。此时，由于内部的输出状态被破坏，模型将不能推理出答案“肉色”，见图 5.13。

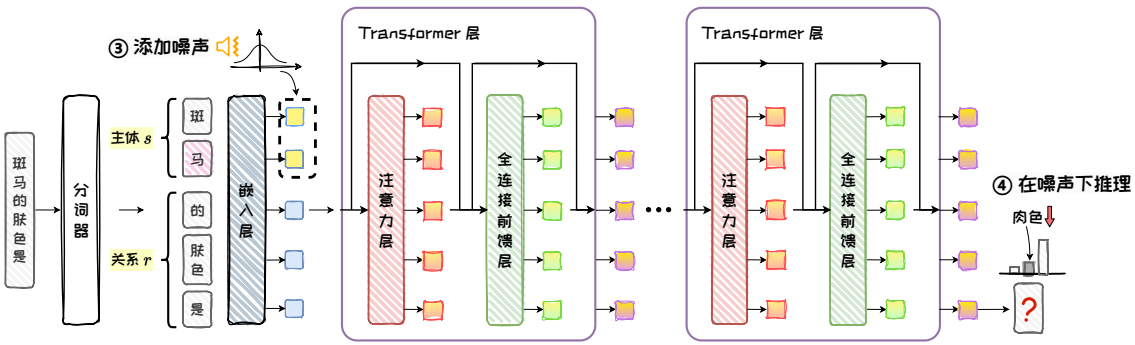


图 5.13: 干扰推理。

最后，对” 斑马的肤色是” 的每个 Token 在每一层的输出向量，分别独立地恢复为正常推理时的值，再次进行推理，记录结果中答案” 肉色” 的概率变化，作为该位置的因果效应强度。如图 5.14，当恢复“马” 这个 Token 在某个 Transformer 层的输出向量时，其右下方蓝色区域的计算都会被影响，从而使输出概率发生变化。此外，ROME 对图中的 Transformer 层（紫色）、全连接前馈层（绿色）、注意力层（红色）三种模块输出都进行了干扰恢复实验并统计了因果效应。

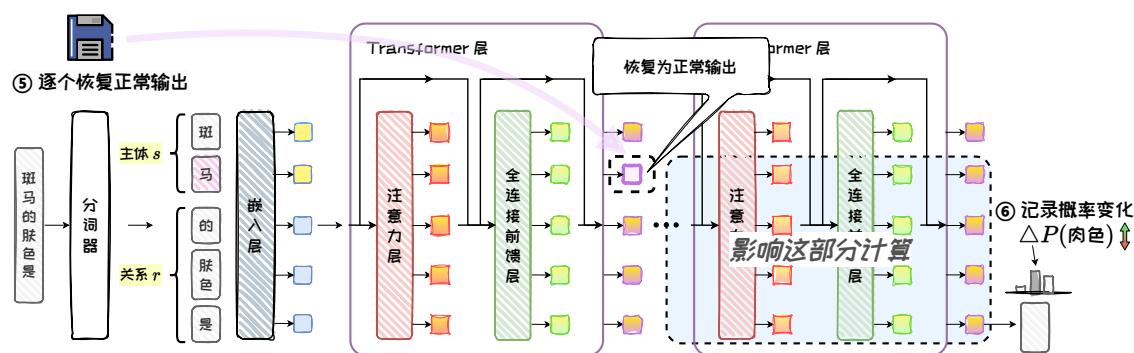


图 5.14: 恢复推理。

ROME 在 1000 个知识陈述上分别对三种模块进行因果跟踪，实验结果揭示了一个新的发现：模型的中间层 Transformer 在处理 s 的最后一个 Token $s^{(-1)}$ （如示例中的“马”）时，表现出显著的因果效应。尽管模型的末尾层 Transformer 在处理 q 的最后一个 Token $q^{(-1)}$ 时，也具有很强的因果效应，但由于这部分内部状态靠近模型输出，因此这一结果并不令人意外。进一步地，对比全连接前馈层和注意力层的因果效应，ROME 发现中间层 Transformer 在处理 $s^{(-1)}$ 时的因果效应主要来自全连接前馈层。而注意力层主要对末尾层 Transformer 处理 $q^{(-1)}$ 产生贡献。基于这些发现，ROME 认为模型中间层的全连接前馈层可能是模型中存储知识的关键位置。

2. 阻断实验

为了进一步区分全连接前馈层和注意力层在 $s^{(-1)}$ 处的因果效应中所起到的作用，并且验证全连接前馈层的主导性，ROME 修改了恢复推理中的计算路径，对两

种模型结构进行了阻断实验。具体来说，在恢复某一层 Transformer 处理 $s^{(-1)}$ 的输出后，将后续的全连接前馈层（或注意力层）冻结为干扰状态，即隔离后续的全连接前馈层（或注意力层）计算，然后观察模型性能的下陷程度，如图 5.15。通过这种方法，能够明确全连接前馈层在模型性能中的关键作用。

比较阻断前后的因果效应，ROME 发现如果没有后续全连接前馈层的计算，中间层在处理 $s^{(-1)}$ 时就会失去因果效应，而末尾层的因果效应几乎不受全连接前馈层缺失的影响。而在阻断注意力层时，模型各层处理 $s^{(-1)}$ 时的因果效应只有较小的下降。

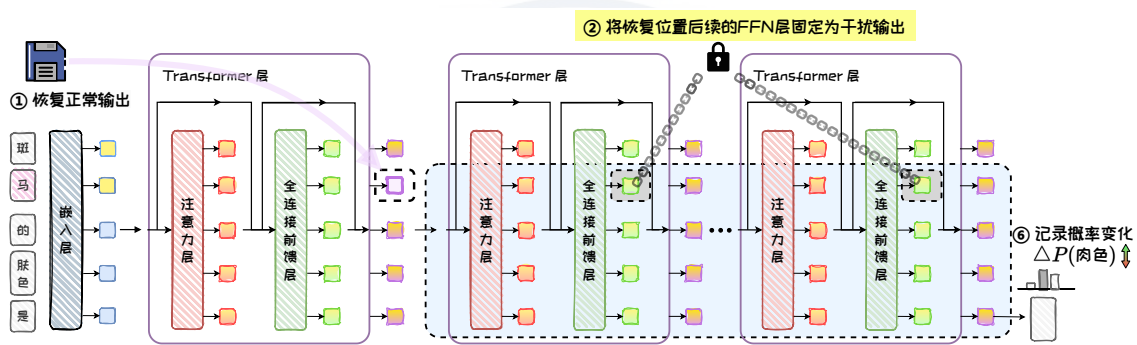


图 5.15: 阻断实验。

基于上述因果跟踪及阻断实验的结果，ROME 认为在大语言模型中，知识存储于模型的**中间层**，其关键参数位于**全连接前馈层**，而且特定中间层的全连接前馈层在处理主体的**末尾 Token** 时发生作用。

5.4.2 知识存储机制

明确了知识存储的位置之后，自然引出下一个关键问题：大语言模型具体是如何存储这些知识的？只有了解知识存储的机制，才能有效地设计编辑方法。基于知识定位的实验结果以及过去的相关研究，ROME 汇总了现有的观点，对知识存储机制做出了合理的假设。

当前,针对大语言模型知识存储机制,研究人员提出了众多观点。Geva 等人 [8] 认为全连接前馈层可以被看作键值存储体,用以存储知识,这与因果跟踪的实验结果一致。Elhage 等人 [7] 指出自注意力机制具有信息复制的作用,每个注意力头都可以被理解为独立的运算单元,其计算结果被添加到残差流中。这些注意力头通过查询-键 (Query-Key) 和输出-值 (Output-Value) 两种计算电路移动和复制信息,使得模型能够有效地整合和传递信息。此外,Zhao 等人 [29] 发现在 Transformer 架构中,不同层的位置可以互换,但模型的性能和输出结果不会发生显著变化。这说明多层 Transformer 结构是灵活的,其不同层次的计算具有相似的功能。

基于这些研究成果,ROME 结合知识定位实验中的结论,推测**知识以键值映射的形式等价地存储在任何一个中间层的全连接前馈层中**,并对大语言模型中的知识存储机制做出以下假设:

- 首先,起始的 Transformer 层中的注意力层收集主体 s 的信息,将其汇入至主体的最后一个 Token 的向量表示中。
- 接着,位于**中间层的全连接前馈层对这个编码主体的向量表示进行查询**,将查询到的相关信息融入残差流 (Residual Stream)¹ 中。
- 最后,末尾的注意力层捕获并整理隐藏状态中的信息,以生成最终的输出。

5.4.3 精准知识编辑

在深入探讨了知识存储的位置和机制之后,我们对模型内部的知识存储和回忆有了更清晰的认识。这种洞察不仅提供了一个宏观的视角来观察知识如何在模型中流动和存储,也为具体的知识编辑方法提供了必要的理论基础。在此基础上,本节将详细介绍 ROME 模型编辑方法,展示如何对模型内部参数进行调整和优化,以实现精准的模型知识编辑。

¹残差流 (Residual Stream) 是指通过残差连接在神经网络层之间传播的信息流。可以想象注意力层和全连接前馈层分别以不同方式向残差信息流中更新信息。

与 T-Patcher 相似，ROME 同样将全连接前馈层视为一个键值存储体。但不同的是，T-patcher 将上投影矩阵的参数向量看作键向量，将下投影矩阵的参数向量看作值向量，而 ROME 则是将下投影矩阵的输入向量看作键向量，将其输出向量看作值向量。具体地，ROME 认为上投影矩阵 W_{fc} 和激活函数 σ 能够计算出用于查询的键向量 k^* ，而下投影矩阵 W_{proj} 会与键向量运算并输出值向量 v^* ，类似信息的查询。为了实现有效的模型编辑，ROME 通过因果跟踪实验定位出一个存储知识的全连接前馈层，然后确定知识在编辑位置的向量表示，最后求解一个约束优化问题得到 W_{proj} 的更新矩阵，从而向全连接前馈层中插入新的键值对。所以，在定位出编辑位置后，ROME 编辑方法主要包括三个步骤：1. 确定键向量；2. 优化值向量；3. 插入知识。

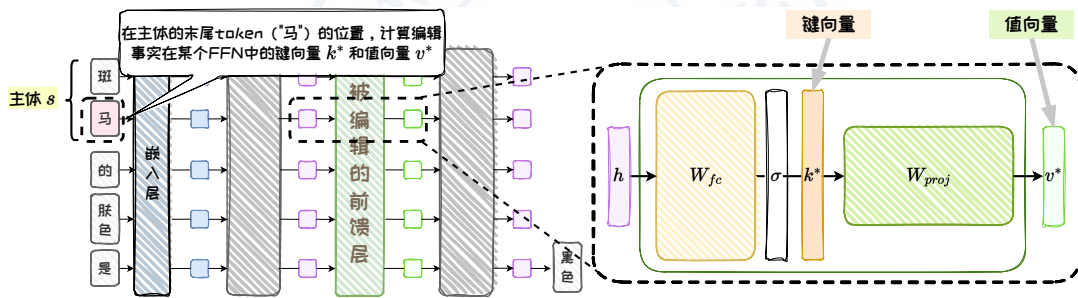


图 5.16: ROME 模型编辑方法。

1. 确定键向量

首先，需要获取 s 在模型内部的向量表示。更准确地说，根据对知识存储机制的假设，需要确定 $s^{(-1)}$ 在被编辑的全连接前馈层中的向量表示。这个向量被称为键向量 k^* ，是 $s^{(-1)}$ 在全连接前馈层中经过激活函数后的输出，它应该编码着 s 。为了确定 k^* ，ROME 将 s 输入模型，直接读取 $s^{(-1)}$ 在激活函数后的向量表示作为 k^* 。而且，为确保 k^* 的泛化性，会在 s 前拼接随机的不同前缀文本进行多次推理，计算平均的向量表示作为 k^* 。见图 5.17。

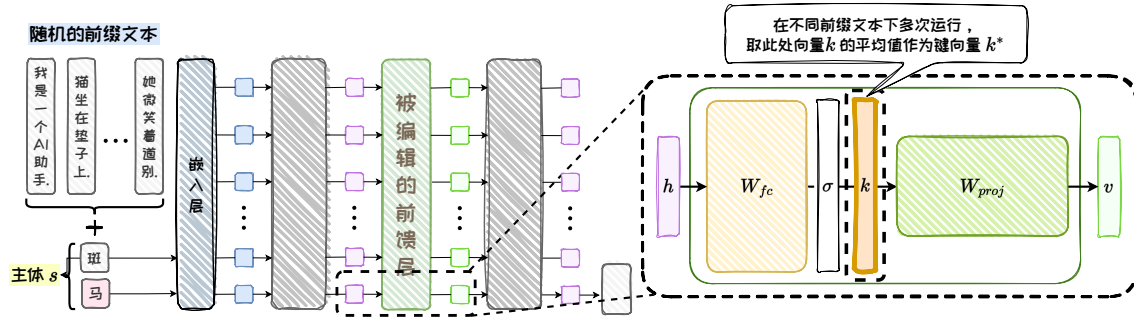


图 5.17: 确定键向量。

键向量的计算公式如下：

$$k^* = \frac{1}{N} \sum_{j=1}^N k(x_j + s), \quad (5.15)$$

其中， N 为样本数量， j 为前缀文本索引， x_j 为随机前缀文本； $k(x_j + s)$ 代表在拼接前缀文本 x_j 时， s 的末尾 Token 在被编辑的全连接前馈层中的激活函数输出，即 W_{proj} 的输入。

2. 优化值向量

然后，需要确定一个值向量 v^* ，作为 W_{proj} 与 k^* 运算后的期望结果，即全连接前馈层处理 $s^{(-1)}$ 的期望输出，它应该将 (r, o) 编码为 s 的属性。ROME 通过优化全连接前馈层的输出向量获得 v^* 。在训练过程中，ROME 通过设计损失函数 $\mathcal{L}(v) = \mathcal{L}_1(v) + \mathcal{L}_2(v)$ 以确保编辑的准确性和局部性，如图 5.18。其中 v 是优化变量，用于替换全连接前馈层的输出。

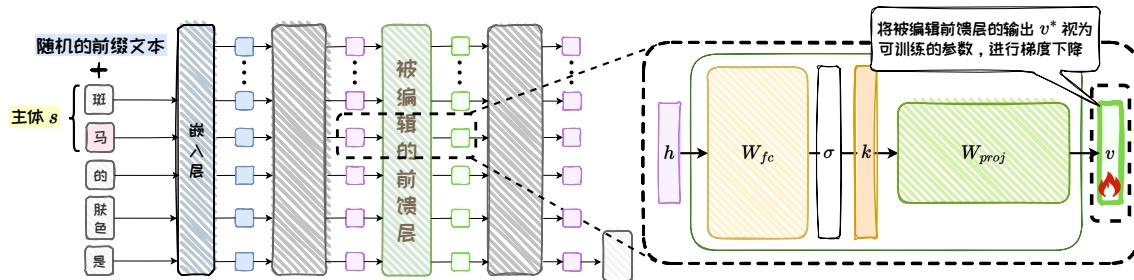


图 5.18: 优化值向量。

损失函数 $\mathcal{L}(v)$ 的公式如下：

$$\mathcal{L}(v) = \mathcal{L}_1(v) + \mathcal{L}_2(v) \quad (5.16)$$

$$\mathcal{L}_1(v) = \frac{1}{N} \sum_{j=1}^N -\log \mathbb{P}_{M'}(o \mid x_j + p) \quad (5.17)$$

$$\mathcal{L}_2(v) = D_{KL}(\mathbb{P}_{M'}(x \mid p') \parallel \mathbb{P}_M(x \mid p')), \quad (5.18)$$

其中， M 为原始模型； M' 为优化 v 时的模型； o 为客体，即目标答案； p 为所编辑的目标问题 prompt； D_{KL} 为 KL 散度； p' 是有关 s 的含义的 prompt，例如“斑马是”。

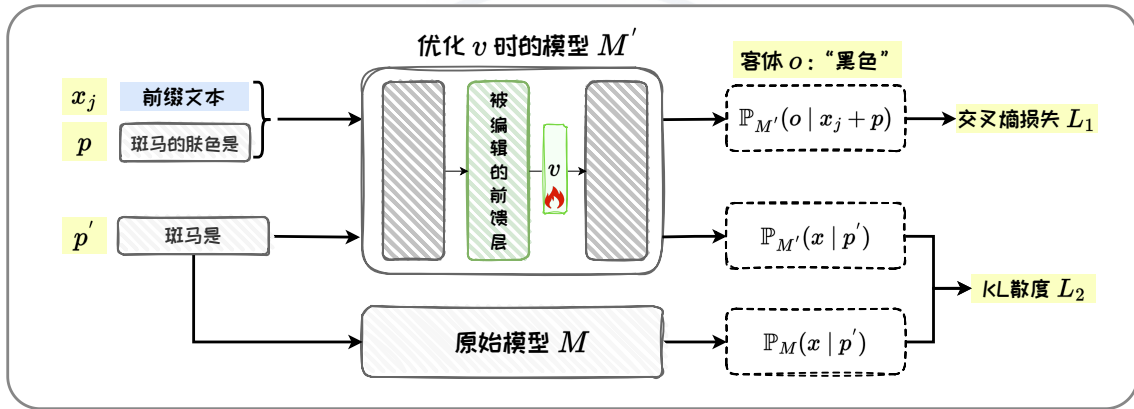


图 5.19: 值向量损失函数。

如图 5.19, 在 $\mathcal{L}(v)$ 中，为了确保准确性， $\mathcal{L}_1(v)$ 旨在最大化 o 的概率，通过优化 v 使网络对所编辑的问题 prompt p 做出正确的预测，与计算 k^* 时相同，也会在 p 之前拼接不同前缀文本；为了确保局部性， $\mathcal{L}_2(v)$ 在 $p' = \{s\}$ 是”这种 prompt 下，最小化 M' 与 M 输出的 KL 散度，以避免模型对 s 本身的理解发生偏移，从而确保局部性。

3. 插入知识

确定了知识在编辑位置的向量表示 k^* 和 v^* 之后，ROME 的目标是调整全连接前馈层中的下投影矩阵 W_{proj} ，使得 $W_{proj}k^* = v^*$ ，从而将新知识插入到全连接

前馈层中。然而，在插入新知识的同时，需要尽量避免影响 W_{proj} 中的原有信息。因此，ROME 将这一问题建模为一个带约束的最小二乘问题，通过求解 W_{proj} 的更新矩阵，将键值向量的映射插入该矩阵，同时不干扰该层中已有的其他信息。由于在求解时， W_{proj} 的更新矩阵的秩为一，因此该方法称作秩一模型编辑。

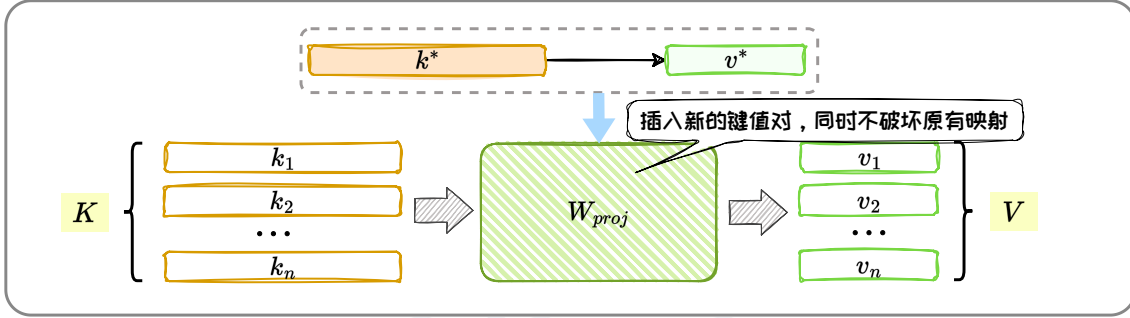


图 5.20: 插入新的键值对。

具体来说，ROME 将 W_{proj} 视为一个线性的键值存储体，即 $WK \approx V$ ，其中编码着键向量集 $K = [k_1, k_2, \dots, k_n]$ 与值向量集 $V = [v_1, v_2, \dots, v_n]$ 的映射。ROME 的目标是在向 W_{proj} 添加新的键值对 (k^*, v^*) 的前提下，不破坏现有的映射关系，见图 5.20。该过程可抽象为一个带约束的最小二乘问题，其形式如下：

$$\min \|\hat{W}K - V\| \quad (5.19)$$

$$\text{s.t. } \hat{W}k^* = v^*. \quad (5.20)$$

该问题可推导出闭式解为：

$$\hat{W} = W + \Lambda(C^{-1}k^*)^T, \quad (5.21)$$

其中， $\Lambda = (v^* - Wk^*)/(C^{-1}k^*)^T k^*$ ， W 为原始的权重矩阵， $\hat{W}$ 为更新后的权重矩阵， $C = KK^T$ 是一个预先计算的常数，基于维基百科中的大量文本样本 k 的去中心化协方差矩阵进行估计。利用这一简洁的代数方法，ROME 能够直接插入代表知识元组的键值对 (k^*, v^*) ，实现对模型知识的精确编辑。

ROME 能够通过因果跟踪精确定位并编辑与特定事实关联的中层前馈模块，